

MPC-PiNNs technique applied to Quadrotors

Elective in Robotics [UAV] Course Project

Master's Degree in Artificial Intelligence and Robotics

Authors (matricole)

Academic Year 2024/2025



SAPIENZA
UNIVERSITÀ DI ROMA



Table of Contents

1 Introduction

- ▶ Introduction
- ▶ Robot Models
- ▶ Model Predictive Control
- ▶ Neural Network
- ▶ Physical Informed Neural Networks
- ▶ Results and Problems Encountered
- ▶ Conclusions



Neural Model Predictive Control (NMPC)

1 Introduction

In this work we have tried to combine Model Predictive Control along with PiNNs to trajectory tracking problem applied to quadrotors. We have used the Unicycle robot as simple test case for our experiments.



Table of Contents

2 Robot Models

- ▶ Introduction
- ▶ **Robot Models**
- ▶ Model Predictive Control
- ▶ Neural Network
- ▶ Physical Informed Neural Networks
- ▶ Results and Problems Encountered
- ▶ Conclusions



Quadorotor

2 Robot Models

Since we have used the Crazyflie robot as reference, instead of using the classic second order simplified model we have used a first order one.

$$\begin{cases} \ddot{x} = -(\cos \psi \sin \theta \cos \phi + \sin \psi \sin \phi) \frac{T}{m} \\ \ddot{y} = -(\sin \psi \sin \theta \cos \phi - \cos \psi \sin \phi) \frac{T}{m} \\ \ddot{z} = -(\cos \theta \cos \phi) \frac{T}{m} + g \\ \ddot{\phi} = \frac{\tau_{\phi}}{I_x} \\ \ddot{\theta} = \frac{\tau_{\theta}}{I_y} \\ \ddot{\psi} = \frac{\tau_{\psi}}{I_z} \end{cases} \quad (1)$$



Table of Contents

3 Model Predictive Control

- ▶ Introduction
- ▶ Robot Models
- ▶ **Model Predictive Control**
- ▶ Neural Network
- ▶ Physical Informed Neural Networks
- ▶ Results and Problems Encountered
- ▶ Conclusions



Model Predictive Control

3 Model Predictive Control

Our objective is to minimize at each step the error with respect to the reference trajectory, while also maintaining the module of the input actions as low as possible.

$$\min_{u(0), u(1), \dots, u(N-1)} \sum_{k=0}^{N-1} \ell(x(k), u(k)) + \ell_f(x(N)) \quad (2)$$

$$\text{s.t. } x(k+1) = f(x(k), u(k)), \quad k = 0, \dots, N-1, \quad (3)$$

$$x(k) \in \mathcal{X}, \quad u(k) \in \mathcal{U}, \quad k = 0, \dots, N-1, \quad (4)$$

$$x(0) = x_0 \quad (5)$$

This approach leads to good result for both the drone and unicycle models.



Table of Contents

4 Neural Network

- ▶ Introduction
- ▶ Robot Models
- ▶ Model Predictive Control
- ▶ **Neural Network**
- ▶ Physical Informed Neural Networks
- ▶ Results and Problems Encountered
- ▶ Conclusions



Simple MLP

4 Neural Network

The architecture is characterized by two hidden layers of 128 neurons and takes as input the current state and the action that I want to apply.

$$\dot{x} = f_{\theta}(x, u) \quad (6)$$

$$loss = \frac{1}{k} \sum_{i=t}^{t+k} (\bar{s}_i - s_i)^2 \quad (7)$$

The loss is computed as the MSE between the neural and the nominal evolutions of the state.

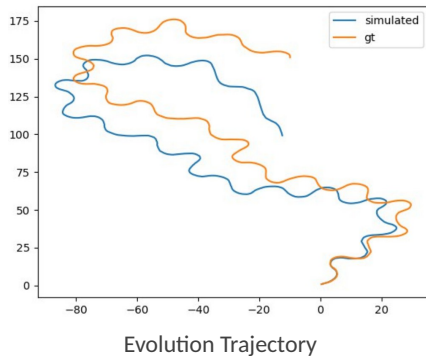
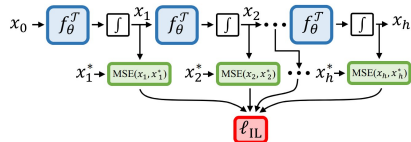




Table of Contents

5 Physical Informed Neural Networks

- ▶ Introduction
- ▶ Robot Models
- ▶ Model Predictive Control
- ▶ Neural Network
- ▶ **Physical Informed Neural Networks**
- ▶ Results and Problems Encountered
- ▶ Conclusions



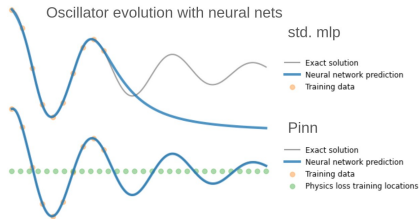
PiNNs

5 Physical Informed Neural Networks

Physical Informed Neural Networks (PiNNs) propose an alternative approach where the solution is approximated by a neural network that is trained not only on data but also by enforcing the underlying physical laws via the differential equations.

$$\mathcal{L}_{\text{data}}(\theta) = \frac{1}{N_u} \sum_{i=1}^{N_u} |u_{\theta}(\mathbf{x}_i, t_i) - u_i^{\text{obs}}|^2, \quad (8)$$

$$\mathcal{L}_{\text{phys}}(\theta) = \frac{1}{N_r} \sum_{j=1}^{N_r} |\mathcal{N}[u_{\theta}](\mathbf{x}_j, t_j)|^2, \quad (9)$$



PiNN vs Classical MLP

Problems and Limitations:

- Cannot handle control input
- Fixed initial condition



PiNN with Controls

5 Physical Informed Neural Networks

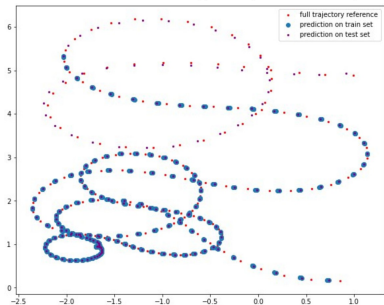
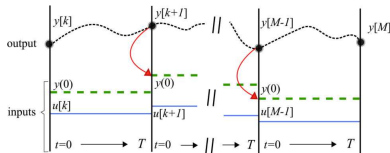
The Pinn-C modifies the formulation of the pinn, by dividing the evolution into sub-transitions with constant input. The network takes as input $(x(o), \bar{u}, t)$.

Pros:

- it can handle larger time-steps with respect to the original formulation
- can accept arbitrary starting conditions

Cons:

- relies heavily on the dataset



Trajectory with PiNN-C



Table of Contents

6 Results and Problems Encountered

- ▶ Introduction
- ▶ Robot Models
- ▶ Model Predictive Control
- ▶ Neural Network
- ▶ Physical Informed Neural Networks
- ▶ **Results and Problems Encountered**
- ▶ Conclusions



Dataset Generation

6 Results and Problems Encountered

We have tried different strategies:

- random velocity commands over the horizon
- random velocity commands constant over the horizon



Integration technique used

6 Results and Problems Encountered



Why we have chosen to use PiNNs with Control instead of classic PiNNs

6 Results and Problems Encountered



Drift Problem

6 Results and Problems Encountered

After some training, we have tested our neural model along with the MPC with nice results (check video). However, we found out that by having zero control inputs the networks still moves the state somewhere else. We have tried to overcome this problem in some ways but with poor results:

- Sample N sample points at the boundary during each epoch
- Add a penalty term related to the boundary condition



Possible Local Minima

6 Results and Problems Encountered

During each training experiment, we have found that all losses behave correctly but they remain stuck at some points even with few iterations. Also in this case we have tried extensively hyperparameter tuning without any solution to the problem.

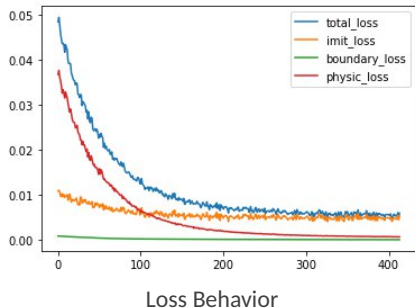




Table of Contents

7 Conclusions

- ▶ Introduction
- ▶ Robot Models
- ▶ Model Predictive Control
- ▶ Neural Network
- ▶ Physical Informed Neural Networks
- ▶ Results and Problems Encountered
- ▶ **Conclusions**